C2 High Performance Computing Coursework Report

Baiyuan Chen

CRSID: bc654

bc654@cam.ac.uk

Word Count: 2098

MPhil in Data Intensive Science

University of Cambridge

0. Introduction

This report describes the incremental development of an OpenMP-parallelized Cholesky factorization routine for symmetric positive-definite matrices. All benchmarks were run on the CSD3 **icelake** partition (Intel Xeon Platinum 8360Y, 76 cores per node) unless otherwise stated. The code is compiled with `gcc -O3 -march=native -funroll-loops -fopenmp`.

0.1. Project structure.

The repository is organized as follows:

- `src/` — library source files (`cholesky_baseline.c`, ..., `cholesky_omp3.c`).
- `include/` — public header `mphil_dis_cholesky.h`.
- `example/` — example program (`main.c`) demonstrating usage of the library.
- `test/` — correctness tests and benchmarking infrastructure.
- `report/` — this report.

0.2. Testing and correctness verification.

The test suite (`test/test_cholesky.c`) verifies correctness via:

(1) Hand-computed reference cases (the 2×2 and 3×3 examples from the specification).
(2) Frobenius-norm reconstruction error: $\|LL^T - C\|_F / \|C\|_F < 10^{-10}$.
(3) Comparison of $\log |C|$ against LAPACK's `dpotrf` at $n = 100, 500, 1000$. LAPACK is used *only* in the test harness as a reference; the library itself uses no external libraries beyond OpenMP.

Tests are built and run via `make IMPL=omp3 test`, which links the test binary against `-llapack -lblas` (available system-wide on CSD3).

0.3. Benchmarking.

All benchmarks are automated via two scripts:

- `test/run_bench_all.sh` — builds and benchmarks all implementations, writing timing and $\log |C|$ results to `test/bench_results.csv`. It also runs a thread-scaling sweep of omp3 at thread counts 1, 2, 4, 8, 16, 32, 64, writing results to `test/bench_scaling.csv`.
- `test/plot_bench.py` — reads the CSV files and produces three plots: execution time vs. matrix size, accuracy vs. LAPACK, and thread scaling. Requires Python 3 with `matplotlib`.

To reproduce all figures:

```
bash test/run_bench_all.sh 64
python3 test/plot_bench.py
```

1. Optimization Strategy

Baseline and three optimized versions (opt1, opt2, opt3) are developed incrementally.

1.1. Baseline.

The baseline implementation follows the algorithm provided in the coursework specification directly. For each pivot $p = 0, 1, \ldots, n-1$, it performs three steps:

(1) **Diagonal update:** $c_{pp} \leftarrow \sqrt{c_{pp}}$
(2) **Row and column scaling:** $c_{pj} \leftarrow c_{pj}/c_{pp}$ and $c_{ip} \leftarrow c_{ip}/c_{pp}$ for all $i, j > p$.
(3) **Rank-1 trailing update:** $c_{ij} \leftarrow c_{ij} - c_{ip} \cdot c_{pj}$ for all $i, j > p$.

The trailing update dominates the cost. Since both the inner and outer loops of the update iterate over $O(n-p)$ elements, each step is $O((n-p)^2)$, and summing over all pivots gives a total complexity of $O(n^3/3)$. In the baseline, the inner loops are ordered with j outer and i inner:

```
for (int j = p + 1; j < n; j++)
    for (int i = p + 1; i < n; i++)
        c[i*N + j] -= c[i*N + p] * c[p*N + j];
```

This ordering is suboptimal for row-major storage: the inner loop strides over `c[i*N+j]` with stride N (one cache line per iteration), causing frequent cache misses.

1.2. **opt1: Loop reordering and micro-optimizations.** Three changes are introduced in opt1:
Loop swap. The inner loops of the trailing update are reordered so that i is outer and j is inner:

```
for (int i = p + 1; i < n; i++) {
    double c_ip = c[i*N + p];      // hoisted invariant
    for (int j = p + 1; j < n; j++)
        c[i*N + j] -= c_ip * c[p*N + j];
}
```

Now the inner loop accesses `c[i*N+j]` and `c[p*N+j]` with stride 1, so consecutive iterations hit the same cache line. For a 64-byte cache line holding 8 doubles, this reduces the number of cache line fetches by up to 8× compared to the baseline's stride-N access.
Invariant hoisting. The value `c[i*N+p]` is constant throughout the inner j-loop, so it is loaded once into a register (`c_ip`), avoiding a redundant memory access per inner iteration.
Reciprocal multiply. The row and column scaling steps replace division by `diag` with multiplication by `1.0/diag`. On modern x86 microarchitectures, `MULSD` has a throughput of 1 cycle vs. 4–6 cycles for `DIVSD`, giving a 4–6× speedup on this operation. The cost of computing the reciprocal once (∼20 cycles) is amortized over $O(n)$ multiplications.

1.3. **opt2: Batched rank-1 updates (blocked panel factorization).** In opt1, each pivot p applies a rank-1 update to the trailing submatrix of size $(n-p-1)^2$. For large n, this submatrix far exceeds cache capacity, so every rank-1 update streams the entire trailing matrix from main memory. Over n pivots, the total memory traffic is $O(n^3)$ doubles.

The fix is to *delay* the trailing update. opt2 groups $N_B = 64$ consecutive pivots into a *panel* and applies all their updates in one pass—reducing memory traffic by a factor of $\sim N_B$. Each panel is processed in three phases (illustrated in Figure 1 for $n = 12$, $N_B = 4$):
Phase A (panel factorization): run the unblocked opt1 algorithm on just the N_B columns of the panel, factoring the diagonal block and scaling the column panel below it. The j-loop stays within the panel boundary and the trailing matrix is not touched.
Phase B (upper row solve): rows pp through $pp+N_B-1$ still have stale values to the right of the panel. For each such row, contributions from earlier panel rows are subtracted and the result is scaled by the diagonal (the same treatment opt1 would have applied). This is $O(N_B^2 \cdot m)$, small compared to Phase C.
Phase C (trailing update): apply a single rank-N_B update, combining all 64 pivots into one pass:

```
for (int i = pp + pb; i < n; i++) {
    for (int k = pp; k < pp + pb; k++) {
        double c_ik = c[i*N + k];
        for (int j = pp + pb; j < n; j++)
```

```
            c[i*N + j] -= c_ik * c[k*N + j];
    }
}
```

When the CPU loads row i into cache, the k-loop runs 64 times before moving on. The panel data ($N_B \times 8 = 512$ bytes) fits in L1, so the trailing matrix passes through memory n/N_B times instead of n times. $N_B = 64$ is chosen to fit within the 48 KB L1 cache of the Icelake core.

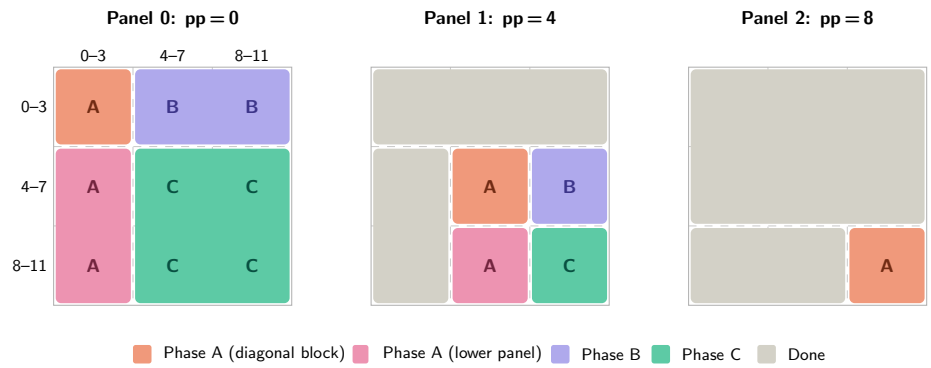

FIGURE 1. Blocked Cholesky phase map for $n = 12$, $N_B = 4$. Phase C (trailing update) dominates the cost and shrinks each iteration. By the final panel only Phase A remains.

1.4. **opt3: Lower-triangle-only trailing update.** In opt2, Phase C updates the *full* trailing submatrix, but since C is symmetric, only the lower triangle is actually needed. The upper triangle is just a copy of the lower, and Phase B of the next panel will overwrite it anyway. This means roughly half the FLOPs in Phase C are wasted.

opt3 makes three changes to exploit this:

(i) Phase C restricted to the lower triangle. The inner j-loop bound changes from `j < n` to `j <= i`:

```
for (int i = pp + pb; i < n; i++)
    for (int k = pp; k < pp + pb; k++) {
        double c_ik = c[i*N + k];
        for (int j = pp + pb; j <= i; j++)
            c[i*N + j] -= c_ik * c[k*N + j];
    }
```

The total FLOP count for Phase C drops from $m^2 \cdot N_B$ to $\frac{1}{2}m^2 \cdot N_B$, nearly doubling throughput for large m.

(ii) Phase B replaced by transpose copy. Since Phase C now only writes the lower triangle, the upper-triangular entries c_{kj} for $k < pp + N_B \leq j$ needed by the next Phase C are simply $L^T_{kj} = L_{jk} = c_{jk}$. The entire triangular solve is replaced by a copy:

```
for (int k = pp; k < pp + pb; k++)
    for (int j = pp + pb; j < n; j++)
        c[k*N + j] = c[j*N + k];
```

This replaces $O(N_B^2 \cdot m)$ FLOPs with $O(N_B \cdot m)$ memory copies, eliminating Phase B as a cost centre entirely.

(iii) Mirror step. Before each Phase A, the lower triangle of the current $N_B \times N_B$ diagonal block is copied to its upper triangle, since the previous Phase C only wrote the lower entries:

```
if (pp > 0)
    for (int i = pp + 1; i < pp + pb; i++)
        for (int j = pp; j < i; j++)
            c[j*N + i] = c[i*N + j];
```

This is $O(N_B^2)$ per panel — negligible compared to Phase C.

2. Parallelization Strategy

2.1. **omp1: Basic OpenMP parallelization.** omp1 adds OpenMP directives to the opt1 algorithm. A single persistent parallel region wraps the entire pivot loop, avoiding repeated fork/join overhead (which can cost $\sim$10 μs per fork on modern systems):

```
#pragma omp parallel
{
    for (int p = 0; p < n; p++) {
        #pragma omp single
        {   // diagonal update: single scalar op
            double diag = sqrt(c[p*N + p]);
            c[p*N + p] = diag;
            inv_diag = 1.0 / diag;
        }   // implicit barrier

        #pragma omp for schedule(static)
        for (int j = p + 1; j < n; j++)
            c[p*N + j] *= inv_diag;

        #pragma omp for schedule(static)
        for (int i = p + 1; i < n; i++)
            c[i*N + p] *= inv_diag;

        #pragma omp for schedule(static)
        for (int i = p + 1; i < n; i++) {
            double c_ip = c[i*N + p];
            for (int j = p + 1; j < n; j++)
                c[i*N + j] -= c_ip * c[p*N + j];
        }
    }
}
```

The diagonal update is performed by a single thread via `#pragma omp single`; the implicit barrier after the `single` block ensures that all threads observe the updated `inv_diag` before proceeding. The row scaling, column scaling, and trailing update are parallelized over their respective loop indices. Each iteration is independent (no write conflicts), so no atomic operations or critical sections are required. Static scheduling is used because all rows have equal work ($n - p - 1$ multiplications).

2.2. **omp2: Blocked + parallel.** omp2 combines the blocking strategy of opt2 with OpenMP parallelization. The same persistent parallel region encloses the outer panel loop.
Phase A. Within the panel, each pivot k depends on the previous one (the rank-1 update must complete before the next diagonal is computed), so the k-loop is inherently sequential. However, the column scaling below the panel and the rank-1 update on the lower panel are parallelized across threads via `#pragma omp for`:

```
// Scale column below pk --- parallelized
#pragma omp for schedule(static)
for (int i = pk + 1; i < n; i++)
    c[i*N + pk] *= inv;

// Rank-1 update on lower panel --- parallelized
#pragma omp for schedule(static)
for (int i = pp + pb; i < n; i++) {
    double c_ip = c[i*N + pk];
    for (int j = pk + 1; j < pp + pb; j++)
        c[i*N + j] -= c_ip * c[pk*N + j];
}
```

The small intra-block operations (row scaling within the panel, rank-1 update within the diagonal block) are executed by a single thread via `#pragma omp single`, since the overhead of distributing $O(N_B^2)$ work across threads would exceed the computation cost.
Phase B. The triangular solve is $O(N_B^2 \cdot m)$, which is small relative to Phase C's $O(m^2 \cdot N_B)$. It is executed by a single thread inside a `#pragma omp single` block.
Phase C. The dominant trailing update is parallelized by distributing rows across threads:

```
#pragma omp for schedule(static)
for (int i = pp + pb; i < n; i++) {
    for (int k = pp; k < pp + pb; k++) {
        double c_ik = c[i*N + k];
        for (int j = pp + pb; j < n; j++)
            c[i*N + j] -= c_ik * c[k*N + j];
    }
}
```

Each row i is fully independent (no shared write targets), so no synchronization is needed within Phase C. Static scheduling is appropriate here because every row performs the same amount of work: $N_B \times m$ multiply-subtract operations.

2.3. **omp3: Guided scheduling for triangular workload.** omp3 builds on omp2 but incorporates the lower-triangle-only optimization from opt3. This introduces a key scheduling challenge.

In Phase C with the lower-triangle restriction, row i performs work proportional to $(i - pp - N_B + 1)$ rather than a constant m. Concretely, with T threads and static scheduling, each thread receives a contiguous chunk of m/T consecutive rows. The thread assigned to the *last* chunk (rows with the largest indices) does:

$$W_{\text{last}} = \sum_{i=m-m/T}^{m-1} i \approx \frac{m^2}{2T}\left(2 - \frac{1}{T}\right) \tag{1}$$

while the ideal per-thread share is:

$$W_{\text{ideal}} = \frac{1}{T}\sum_{i=0}^{m-1} i \;=\; \frac{m^2}{2T} \tag{2}$$

For $T = 4$, the last thread does $\approx 44\%$ of the total work (vs. the ideal 25%), leaving 3 threads idle for nearly half the phase.

To address this imbalance, `schedule(static)` is replaced with `schedule(guided)` in the Phase C loop is effective:

```
#pragma omp for schedule(guided)
for (int i = pp + pb; i < n; i++) {
    for (int k = pp; k < pp + pb; k++) {
        double c_ik = c[i*N + k];
        for (int j = pp + pb; j <= i; j++)
            c[i*N + j] -= c_ik * c[k*N + j];
    }
}
```

Guided scheduling assigns chunks of size $\max(\lceil R/T \rceil, 1)$ where R is the number of remaining iterations and T is the number of threads. This produces geometrically decreasing chunk sizes: the first chunks are large and cover the early, light rows (low i), while the last chunks are small and cover the late, heavy rows (high i). A thread that receives a large chunk of light rows ends up doing roughly the same total work as a thread that receives a small chunk of heavy rows, naturally balancing the triangular workload.

Moreover, the transpose copy that replaces the triangular solve is also parallelized:

```
#pragma omp for schedule(static)
for (int k = pp; k < pp + pb; k++)
    for (int j = pp + pb; j < n; j++)
        c[k*N + j] = c[j*N + k];
```

Each row k writes to disjoint memory locations, so no synchronization is needed. Static scheduling is used here because all rows perform the same amount of work (m copies each).

3. Performance Results

3.1. **Execution time vs. matrix size.** Figure 2 shows the execution time across all implementations for matrix sizes $n = 100$ to 10,000.

The single-threaded optimizations each build on the last: at n=1000, opt1 is roughly 5× faster than the baseline (0.099 s vs. 0.467 s) from better cache access patterns alone, opt2 brings this down to 0.071 s with blocking, and opt3 reaches 0.037 s by halving the trailing-update work.

Parallelism only pays off once the matrix is large enough. At n=500, opt3 (0.005 s) actually beats omp3 (0.028 s) by over 5×—the matrix is small enough to fit in cache, so the overhead of synchronizing 64 threads (barriers, single-region handoffs, cache-coherence traffic) outweighs any benefit from splitting the work. Beyond $n \approx 1000$ the cubic growth in computation begins to dominate these fixed costs, and by n=10,000 omp3 (1.85 s) is ∼25× faster than opt3 (47.2 s).

Among the parallel variants, the gap between omp2 and omp3 grows with matrix size (4.33 s vs. 1.85 s at n=10,000): the larger the matrix, the worse the triangular load imbalance becomes, and the more guided scheduling and the lower-triangle optimization help.

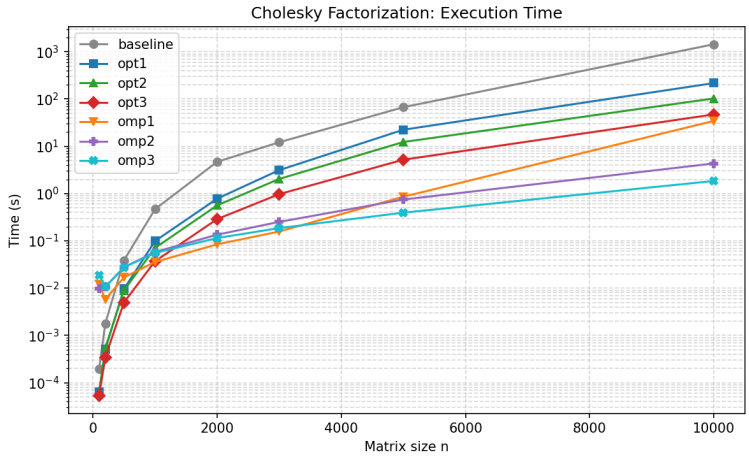

FIGURE 2. Execution time vs. matrix size for all implementations (OMP_NUM_THREADS=64, icelake partition).

3.2. **Accuracy vs. LAPACK reference.** Figure 3 shows the relative error in $\log|C|$ compared to LAPACK's `dpotrf`. All implementations achieve machine-precision accuracy ($\sim 10^{-15}$).

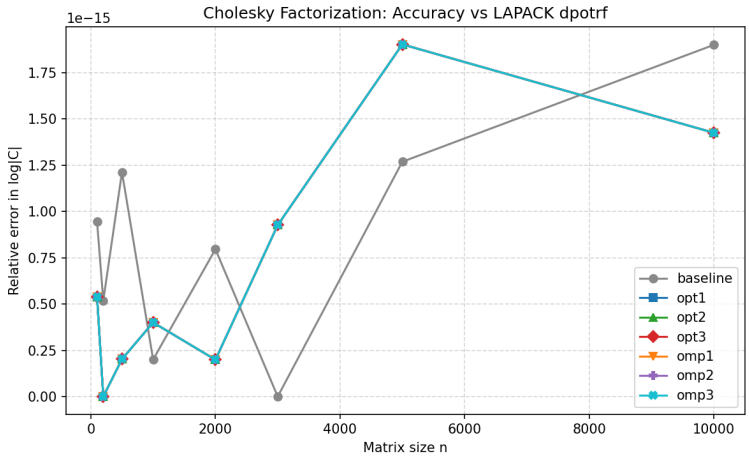

FIGURE 3. Relative error in $\log|C|$ vs. LAPACK `dpotrf` reference.

3.3. **Thread scaling.** Figure 4 shows how omp3 scales with thread count for several matrix sizes.

For small matrices, adding threads makes things worse. At n=100, 64 threads is over 100× slower than one thread (0.019 s vs. 0.0002 s), because the actual computation is tiny compared to the cost of barrier synchronization and thread management. The matrix fits in cache, so there is almost nothing to parallelise.

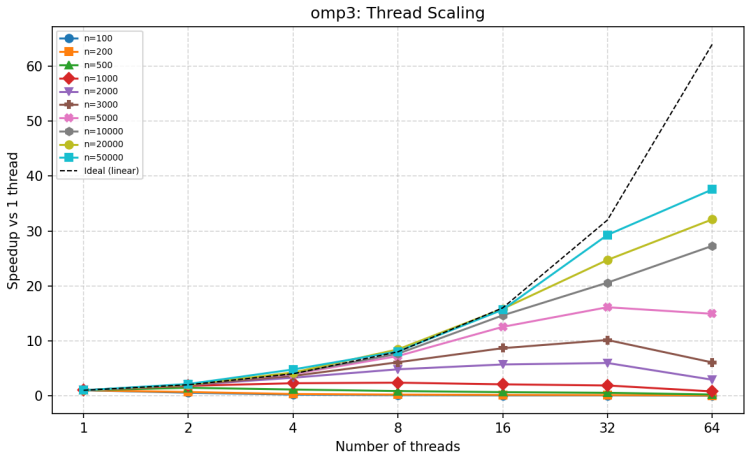

FIGURE 4. omp3 thread scaling on icelake. Each line represents a fixed matrix size.

As n grows and Phase C begins to dominate the runtime, scaling improves steadily. At n=10,000, 64 threads gives a 27× speedup; at n=20,000 this rises to 32×; and at n=50,000 it reaches 37.5×, continuing the trend toward the ideal 64× line. The gap that remains comes from the serial panel factorisation (Phase A), memory bandwidth saturation across 76 cores sharing the same memory controllers, and NUMA effects on the dual-socket node. Still, the trend is clear: the larger the matrix, the closer the scaling gets to linear.

4. Tagged Commits

Table 1 lists the tagged commits corresponding to each significant version of the code.

Tag	**Commit**	**Description**
`baseline-implementation+test-script`	`e2dd2639`	Baseline implementation with tests
`single-threaded-opt1-loop-swap`	`806694b0`	opt1: loop reordering for cache locality
`omp1-loop-swap`	`ba1f3ea8`	omp1: first parallel version
`opt2/omp2`	`97adb319`	Blocked updates, reduced memory traffic
`opt3/omp3`	`4fa64c34`	Lower-triangle update, guided scheduling

TABLE 1. Tagged commits in the Git history.

5. Conclusion

The final implementation (omp3) combines loop reordering for cache locality, blocked panel factorization to reduce memory traffic by $\sim 64\times$, and lower-triangle-only updates to halve Phase C FLOPs, with guided OpenMP scheduling to balance the triangular workload. All implementations match LAPACK's `dpotrf` to machine precision.

Use of generative AI

Claude (Anthropic) was used to assist with code development and report writing. All outputs were reviewed and tested by the author, who takes full responsibility for the final submission.